REPORT · 2026年8月

造像类文物三维建模自动化系统 L1适配层代码说明书

L1 适配层 · 点云 / 网格读写格式适配

文保数字化研发组
2026年8月

目 录

一、概述 3

二、模块划分 4

三、依赖关系与数据流 5

3.1 依赖方向 5

3.2 点云读取数据流 5

四、模块详细设计 6

4.1 pointcloud_io.py — 点云读写 6

4.2 mesh_io.py — 网格读写 6

4.3 __init__.py — 包导出 7

五、接口契约与错误码 8

六、测试与验证 9

一、概述

本文档是《造像类文物三维建模自动化系统（路线 E）》**L1 适配层（io 子包）**的代码说明书。L1 适配层对应工程目录 **src/h3d/io**，职责是「把外部点云 / 网格格式读入为统一数据模型，并把统一数据模型写回外部格式」，同时完成格式与完整性校验。它是整条管线中**唯一**直接依赖Open3D / laspy 的位置，使业务层（preprocess / register / ...）完全不感知具体读写实现。本说明书严格遵循《软件开发项目设计书》第 10 章（五层架构 L1）、《软著方案》2.3.1（适配层抽象原则）、《管线骨架构建分析》第 4~5 章（模块划分与依赖约束）及《开发指南（教学版）》第 14 章的接口约定。

> 设计铁律（《软著方案》2.3.1）：业务代码永远不直接 import open3d / laspy，只调用 io 层提供的抽象接口。这一「适配器模式」带来许可证隔离、可替换性、独创性论证与可测试性四重收益。

二、模块划分

L1 适配层由 3 个文件构成，目录结构与职责如下：

```
src/h3d/io/
+-- __init__.py        # 包导出：统一暴露读写接口（__all__）
+-- pointcloud_io.py   # 点云读写：ply / pcd / xyz / pts / las / laz
+-- mesh_io.py         # 网格读写：obj / ply / stl / glb / gltf
```

模块文件	核心职责	对外关键符号
pointcloud_io.py	点云读写与格式适配（Open3D + laspy）	load_point_cloud, load_all, save_point_cloud, POINTCLOUD_READ_EXT, POINTCLOUD_WRITE_EXT
mesh_io.py	网格读写与格式适配（Open3D）	load_mesh, save_mesh, MESH_READ_EXT, MESH_WRITE_EXT
__init__.py	包导出：统一暴露读写接口	（聚合导出上述符号，供业务层按需导入）

两个读写模块各司其职：**pointcloud_io** 只管点云（输入为统一 Open3D PointCloud），**mesh_io** 只管网格（输入为统一 Open3D TriangleMesh）。二者通过显式导入共享**h3d.core** 的异常与日志，互不交叉。支持格式按扩展名路由，见第四章。

三、依赖关系与数据流

3.1 依赖方向

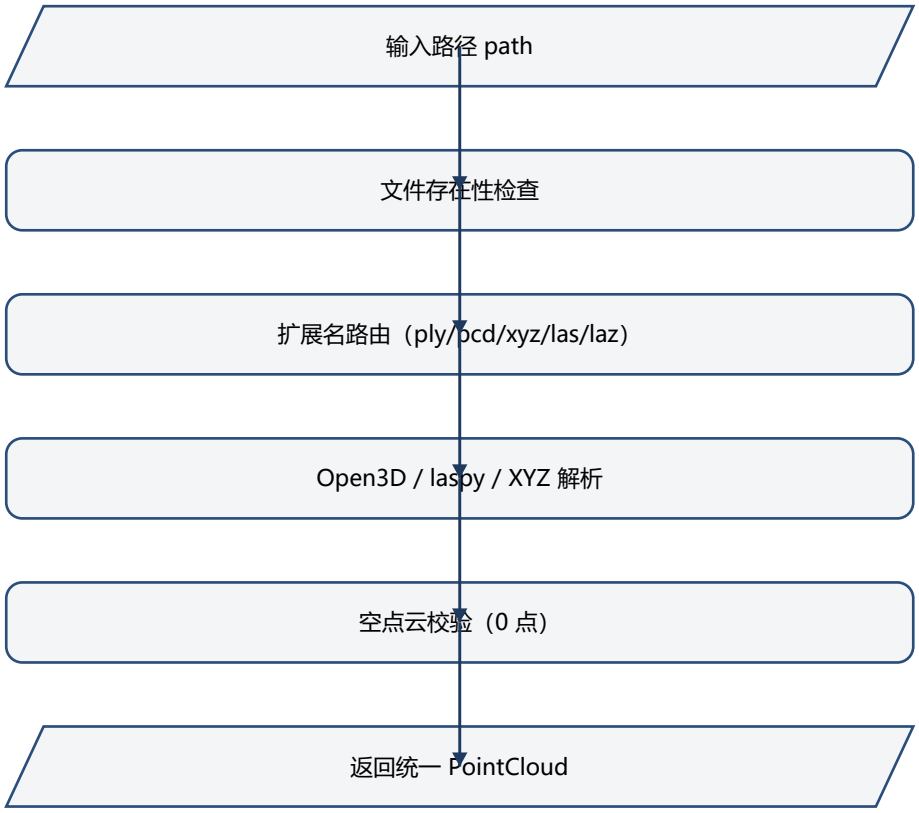
依据《管线骨架构建分析》5.2，io 属 L1 适配层，仅依赖 core 底座（exceptions 与 logging），且不反向（core 不依赖 io），业务层只依赖 io 不直接依赖第三方库：

```
package-level deps (A -> B means A imports B):
  preprocess / register / change / colour / runner / report  -->  io
  io  -->  core.exceptions (InputDataError, ExportError)
  io  -->  core.logging  (get_logger)
  io  -->  Open3D / laspy (仅在读写函数内部惰性导入，适配层独占)
  core  -->  (NONE; does not depend on io)
```

> Open3D / laspy 采用**函数内惰性导入**（_o3d() / _read_las 内 import），使 import h3d.io本身不触发重型三维库加载，业务层单元测试可对 io 层整体 mock，契合《软著方案》「适配层可 mock，业务逻辑单元测试不依赖重型三维库」的可测试性原则。

3.2 点云读取数据流

单文件点云读取（load_point_cloud）的完整流转（自顶向下，非法取值经 InputDataError E100 分支中断）：



load_point_cloud 数据流：缺失 / 不支持格式 / 空点云均抛 InputDataError(E100)

四、模块详细设计

4.1 pointcloud_io.py — 点云读写

点云模块以**扩展名路由**为核心：读取时按后缀分发到 Open3D（ply / pcd）、文本解析（xyz / pts）或 laspy（las / laz），统一归一为 Open3D PointCloud；写出时仅允许 ply / pcd / xyz。所有错误统一转为带错误码的管线异常，不向业务层泄漏第三方库原生异常。

函数	签名	职责
load_point_cloud	load_point_cloud(path) -> PointCloud	读取单文件；缺失/不支持/空点云 → InputDataError(E100)
load_all	load_all(pattern) -> [(path, PointCloud), ...]	glob 批量读取；无匹配 → InputDataError(E100)
save_point_cloud	save_point_cloud(pcd, path) -> None	写回 ply/pcd/xyz；不支持/写失败 → ExportError(E601)
_read_open3d / _read_xyz / _read_las	内部	按格式分发的底层读取实现

```
def load_point_cloud(path: str):
    path = str(path)
    if not os.path.exists(path):
        raise InputDataError(f"点云文件不存在: {path}")
    ext = _ext(path)
    if ext not in POINTCLOUD_READ_EXT:
        raise InputDataError(f"不支持的点云格式 {ext!r}: {path}")
    if ext in _LAS_EXT:
        pcd = _read_las(path)      # laspy(BSD-2)
    elif ext in _TEXT_XYZ_EXT:
        pcd = _read_xyz(path)
    else:
        pcd = _read_open3d(path)  # Open3D(MIT)
    if pcd is None or not pcd.has_points():
        raise InputDataError(f"空点云（0 点）: {path}")
    return pcd
```

操作	支持扩展名	引擎 / 许可证
点云读取	ply, pcd, xyz, pts, las, laz	Open3D(MIT) / laspy(BSD-2)
点云写出	ply, pcd, xyz	Open3D(MIT) / NumPy(BSD-3)

4.2 mesh_io.py — 网格读写

网格模块封装 Open3D 的 TriangleMesh 读写，支持 obj / ply / stl / glb / gltf 读取与 obj / ply / stl / glb 写出（对齐《设计书》M6 标准格式导出）。写 STL 时若网格缺顶点法线，在副本上自动补算，避免污染调用方对象——这是适配层「格式适配」职责的体现。

函数	签名	职责
load_mesh	load_mesh(path) -> TriangleMesh	读取单文件；缺失/不支持/空网格 → InputDataError(E100)
save_mesh	save_mesh(mesh, path) -> None	写回 obj/ply/stl/glb；STL 自动补法线；写失败 → ExportError(E601)

```
def save_mesh(mesh, path: str) -> None:
    ext = _ext(path)
    if ext not in MESH_WRITE_EXT:
```

```
        raise ExportError(f"不支持的网格写出格式 {ext!r}: {path}")
    os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
    # STL 格式要求顶点法线；缺失时在副本上补算，避免污染调用方
    if ext == ".stl" and not mesh.has_vertex_normals():
        mesh = o3d.geometry.TriangleMesh(mesh)
        mesh.compute_vertex_normals()
    if not o3d.io.write_triangle_mesh(path, mesh):
        raise ExportError(f"网格写盘失败: {path}")
```

4.3 __init__.py — 包导出

io/__init__.py 聚合导出两个读写模块的公共符号，业务方只需 **from h3d.io import load_point_cloud, save_point_cloud, load_mesh, save_mesh** 即可，无需关心底层引擎。这是适配器模式的入口契约。

五、接口契约与错误码

io 层对外暴露的稳定接口遵循设计书 13.2 的「函数 + 路径 + 统一数据模型」约定，错误码与设计书 13.3 严格对齐。io 层只抛两类异常：读入类错误用 InputDataError(E100)，写出类错误用 ExportError(E601)。

错误码	异常类	触发场景	处理策略
E100	InputDataError	文件缺失 / 扩展名不支持 / 空点云 / 空网格 / 解析失败	标记跳过，记入 manifest（可恢复）
E601	ExportError	写出格式不支持 / 写盘失败	跳过该格式，记入 manifest（可恢复）

两类异常均继承 **RecoverableError**，供 M0 编排器以 **except RecoverableError** 实现失败隔离——单尊数据坏、单格式不可写，只跳过该尊 / 该格式，不中断队列。

六、测试与验证

L1 适配层配套单元测试位于 **tests/test_io/**，用真实 Open3D / NumPy 生成合成点云与网格，覆盖格式路由、错误路径（E100 / E601）与往返读写，共 20 项，全部通过。

测试文件	覆盖点
test_pointcloud_io.py	ply/pcd/xyz 往返、load_all glob、缺失/空/不支持格式 → E100、写出不支持 → E601、父目录自动创建
test_mesh_io.py	obj/ply/stl 往返、缺失/空/不支持格式 → E100、写出不支持 → E601、STL 自动补法线

```
$ python -m pytest tests/test_io -q
....................                    [100%]
20 passed in 2.72s

$ python -m pytest -q   # 全量（core + io）
.................................................. [100%]
49 passed in 2.55s
```

运行方式：在 .venv_lab 虚拟环境执行 pytest；或 python -c "from h3d.io import load_point_cloud, save_point_cloud" 冒烟验证。核心依赖全部 MIT/BSD（Open3D、NumPy、laspy），GPL 类库一律不进入 io 层。